



coreBuilder Tutorial

coreBuilder — Product Code: 2618-0

Copyright Notice and Proprietary Information

© 2024 Synopsys, Inc. All rights reserved. This Synopsys software and all associated documentation are proprietary to Synopsys, Inc. and may only be used pursuant to the terms and conditions of a written license agreement with Synopsys, Inc. All other use, reproduction, modification, or distribution of the Synopsys software or the associated documentation is strictly prohibited.

Destination Control Statement

All technical data contained in this publication is subject to the export control laws of the United States of America. Disclosure to nationals of other countries contrary to United States law is prohibited. It is the reader's responsibility to determine the applicable regulations and to comply with them.

Disclaimer

SYNOPSYS, INC., AND ITS LICENSORS MAKE NO WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, WITH REGARD TO THIS MATERIAL, INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR PURPOSE.

Trademarks

Synopsys and certain Synopsys product names are trademarks of Synopsys, as set forth at <https://www.synopsys.com/company/legal/trademarks-brands.html>

All other product or company names may be trademarks of their respective owners.

Free and Open-Source Software Licensing Notices

If applicable, Free and Open-Source Software (FOSS) licensing notices are available in the product installation.

Third-Party Links

Any links to third-party websites included in this document are for your convenience only. Synopsys does not endorse and is not responsible for such websites and their practices, including privacy practices, availability, and content.

Synopsys, Inc.
www.synopsys.com

Contents

Revision History.....5

Preface.....7

 Audience..... 7

 Tutorial Organization..... 7

 Related Documentation..... 8

 Web Resources..... 8

 Synopsys Statement on Inclusivity and Diversity..... 8

 Getting Help..... 8

Chapter 1 Introduction to coreBuilder..... 11

 1.1. Important Terms..... 12

 1.2. coreBuilder/coreConsultant Design Flow..... 14

 1.3. System Requirements..... 15

 1.3.1. Required Hardware..... 15

 1.3.2. Required Software..... 15

Chapter 2 coreTools Graphical Interface Fundamentals..... 17

 2.1. Starting coreBuilder..... 18

 2.1.1. Starting coreBuilder in UNIX..... 18

 2.2. Quitting coreBuilder..... 19

 2.3. Using the coreBuilder Console..... 20

 2.3.1. Using the coreBuilder console command line..... 21

Chapter 3 Setting Up the Tutorial..... 23

 3.1. How This Tutorial Is Organized..... 24

 3.2. Creating Tutorial Directories..... 25

 3.2.1. Creating Tutorial Directories in UNIX..... 25

 3.3. Tutorial Directory Contents..... 26

 3.3.1. <Header> src Directory..... 26

 3.3.2. <Header> pkg Directory..... 26

 3.3.3. doc Directory..... 26

Chapter 4 Preparing the Core for Packaging..... 27

4.1. Annotating HDL Source Files.....	28
4.1.1. Adding RTL Configuration Information.....	28
4.1.2. Understanding the Annotations.....	28
Chapter 5 Create a Workspace and Verify Configuration Intent.....	31
5.1. Starting coreBuilder and Creating a Workspace.....	32
5.2. Load Consultant Plugins.....	33
5.3. Load Designs.....	34
5.4. Design Configuration Intent.....	35
5.5. Define Memory Maps.....	36
Chapter 6 Synthesis Setup.....	37
6.1. Specify Clocks and Resets.....	38
6.2. Specify Timing Exceptions.....	39
6.3. Specify Port Constraints.....	40
6.3.1. I/O Timing.....	40
6.4. Verify Budgets.....	41
Chapter 7 Attach Interfaces.....	43
7.1. Load Interface Definitions.....	44
7.2. Add Interfaces.....	45
7.2.1. Port and Parameter Association.....	45
Chapter 8 Packaging.....	47
8.1. Build coreKit.....	48
Appendix A Batch Script.....	49
Appendix B Annotated HDL Source Code.....	53
Appendix C Solution Scripts.....	55
C.1. setup.tcl.....	56
C.2. synthesis.tcl.....	57
C.3. attachInterface.tcl.....	58

Revision History

Version	Date	Description
W-2024.09-1	October 2024	Updated the date and version
W-2024.09	September 2024	Updated the date and version
V-2024.06-2	June 2024	Version update
V-2024.06	June 2024	■ Version and date update
V-2024.03-1	May 2024	■ Version and date update
V-2024.03	March 2024	■ Changed instances of DesignWare Cores to Synopsys IP to align with corporate naming conventions.
V-2023.12-1	January 2024	■ Version and date update
V-2023.12	December 2023	■ Version and date update
V-2023.09-1	September 2023	■ Version update
V-2023.09	September 2023	■ Version and date update
U-2023.06-1	June 2023	■ Version update
U-2023.06	June 2023	■ Version and date update
U-2022.12-2	February 2023	■ Version and date update
U-2022.12	December 2022	■ Version and date update
T-2022.06-SP2	September 2022	■ Version and date update
T-2022.06-SP1	July 2022	■ Version and date update
T-2022.06	June 2022	■ Version and date update

Version	Date	Description
S-2021.09-SP4	April 2022	■ Version and date update
S-2021.09-SP3	March 2022	■ Version and date update
S-2021.09-SP2	January 2022	■ Version and date update
S-2021.09-SP1	October 2021	■ Version and date update
S-2021.09	September 2021	■ Version and date update
L-2016.09-SP1	October 2016	■ Corrected typo in Table 7-1 on page 30 for Interface 2 ■ Updated scripts in the following appendices: □ Appendix A, "Batch Script" □ Appendix B, "Annotated HDL Source Code" □ Appendix C, "Solution Scripts"
L-2016.09	September 2016	■ Version update
K-2015.06-SP7	July 2016	■ Version update
K-2015.06-SP6	June 2016	■ Version update
K-2015.06-SP5-2	April 29, 2016	■ Version update
K-2015.06-SP5-1	April 2016	■ Version update
K-2015.06-SP5	March 2016	■ Version update
K-2015.06	June 2015	■ Version update
J-2014.12	December 2014	■ Various tutorial text updates, screenshots, and coreTools versioning
1.31a	June 2014	■ Updated to match current GUI and latest updates
3.0	January 2002	■ Updated to include GUI interface
1.3a	January 1999	■ Initial creation

Preface

This chapter contains the following sections:

- [“Audience”](#) on page 7
- [“Tutorial Organization”](#) on page 7
- [“Related Documentation”](#) on page 8
- [“Web Resources”](#) on page 8
- [“Synopsys Statement on Inclusivity and Diversity”](#) on page 8
- [“Getting Help”](#) on page 8

Audience

This manual is for engineers who are familiar with high-level design techniques and synthesis using Design Compiler, but not familiar with the coreBuilder suite of tools. A working knowledge of the operating system for your computer and various commands derived from the UNIX operating system is assumed.

Tutorial Organization

The coreBuilder Tutorial explains how to begin using the coreBuilder software. It provides a step-by-step guide to the packaging a piece of core IP.

This tutorial is organized in two major parts, as follows:

Part I helps establish a foundation in the terminology, initial startup, and set-up.

- [Chapter 1, “Introduction to coreBuilder”](#): Terminology, Design Flow and System Requirements.
- [Chapter 2, “coreTools Graphical Interface Fundamentals”](#): Starting coreBuilder, Quitting coreBuilder, and using the coreBuilder console.
- [Chapter 3, “Setting Up the Tutorial”](#): Tutorial Organization, Creating Tutorial Directories, Setting the Path or Alias, and Creating Setup Files.
- [Chapter 4, “Preparing the Core for Packaging”](#): Annotating HDL Source Files.
- [Chapter 5, “Create a Workspace and Verify Configuration Intent”](#): Starting coreBuilder and Creating a Workspace, Loading Consultant Plugins, Loading Designs, and Design Configuration Intent.
- [Chapter 6, “Synthesis Setup”](#): Specify clocks, resets, timing exceptions, and port constraints, and verify budgets.

Part II explains how to run coreBuilder and describes a sample peripheral design used for the tutorial. Then, using the simple design, it guides you through the core packaging flow.

- [Chapter 7, “Attach Interfaces”](#), II: Using coreBuilder: A tutorial session.
- [Chapter 8, “Packaging”](#), II: Using coreBuilder: A tutorial session.
- [Appendix A, “Batch Script”](#), II: Using coreBuilder: A tutorial session.
- [Appendix B, “Annotated HDL Source Code”](#), II: Using coreBuilder: A tutorial session.
- [Appendix C, “Solution Scripts”](#), II: Using coreBuilder: A tutorial session.

Related Documentation

For additional information about the coreBuilder tool, see:

- *coreBuilder User Guide*

Web Resources

- Synopsys IP product information: <https://www.synopsys.com/designware-ip.html>
- Your custom Synopsys IP page: <https://www.synopsys.com/dw/mydesignware.php>
- Documentation through SolvNetPlus: <https://solvnetplus.synopsys.com> (Synopsys password required)
- Synopsys Common Licensing (SCL): <https://www.synopsys.com/keys>

Synopsys Statement on Inclusivity and Diversity

Synopsys is committed to creating an inclusive environment where every employee, customer, and partner feels welcomed. We are reviewing and removing exclusionary language from our products and supporting customer-facing collateral. Our effort also includes internal initiatives to remove biased language from our engineering and working environment, including terms that are embedded in our software and IPs. At the same time, we are working to ensure that our web content and software applications are usable to people of varying abilities. You may still find examples of non-inclusive language in our software or documentation as our IPs implement industry-standard specifications that are currently under review to remove exclusionary language.

Getting Help

If you have a question about using Synopsys products, please consult product documentation that is installed on your network. You can also access documentation for Synopsys IP products on the Web:

- Datasheets for individual Synopsys IP components, located using “Search for IP”:
<https://www.synopsys.com/designware-ip.html>
- License Information -- Synopsys Common Licensing Quickstart:
<http://www.synopsys.com/Support/LI>

You can obtain product support in the following ways:

- Go to SolvNetPlus to submit support requests and access documentation and software downloads:
<https://solvnetplus.synopsys.com/>

- Send an e-mail message to support_center@synopsys.com.
- Global Support Centers -- contact information and hours of operation:
<https://www.synopsys.com/support/global-support-centers.html>

1

Introduction to coreBuilder

The Synopsys coreTools make design reuse a practical and reliable chip design solution by providing automatic configuration and high-quality synthesis of reusable blocks of IP in the IP user's environment. coreBuilder provides a guided flow to capture the designer's knowledge of the design and package that knowledge with the design.

This chapter introduces the coreBuilder/coreConsultant design reuse solution.

This chapter contains the following sections:

- [“Important Terms”](#) on page 12
- [“coreBuilder/coreConsultant Design Flow”](#) on page 14
- [“System Requirements”](#) on page 15

1.1 Important Terms

Before discussing design reuse problems and solutions, it is important to define a few basic terms that will be used throughout this manual:

- **application design**

The core integrator's overall design that contains the reusable core. Typically, an application design includes one or more reusable cores, plus custom, design-specific logic.

- **core**

A block of reusable IP developed by a core developer and used in designs by one or more core integrators.

- **coreAssembler**

The coreAssembler product is part of the complete set of IP reuse tools available from Synopsys. coreAssembler provides a graphical or command-based environment that guides the designer through the assembly and configuration of an IP-based subsystem.

- **coreConsultant**

The coreConsultant utility is part of the complete set of IP reuse tools available from Synopsys. coreConsultant provides a graphical or command environment that guides the user through the installation, configuration, verification, and implementation of the core packaged with coreBuilder.

- **coreKit**

The design package that coreBuilder creates for the core. The coreKit includes the core, the design intent specified by the core developer, and any other files specified by the core developer, such as documentation and/or verification environment files.

- **design context**

Aspects of the target environment of the core within the application design that affect synthesis of the core. The core integrator specifies the design context before coreConsultant generates synthesis scripts for the core. Design context includes such things as wire load model, parent wire load model, and operating conditions.

- **intellectual property (IP)**

In the context of design reuse, we use the term IP to refer to a reusable design. Throughout this manual, IP is assumed to be in the form of a soft core, because coreBuilder and coreConsultant are currently focused on the reuse of soft cores (user-configurable, technology-independent, synthesizable blocks of IP).

- **intent**

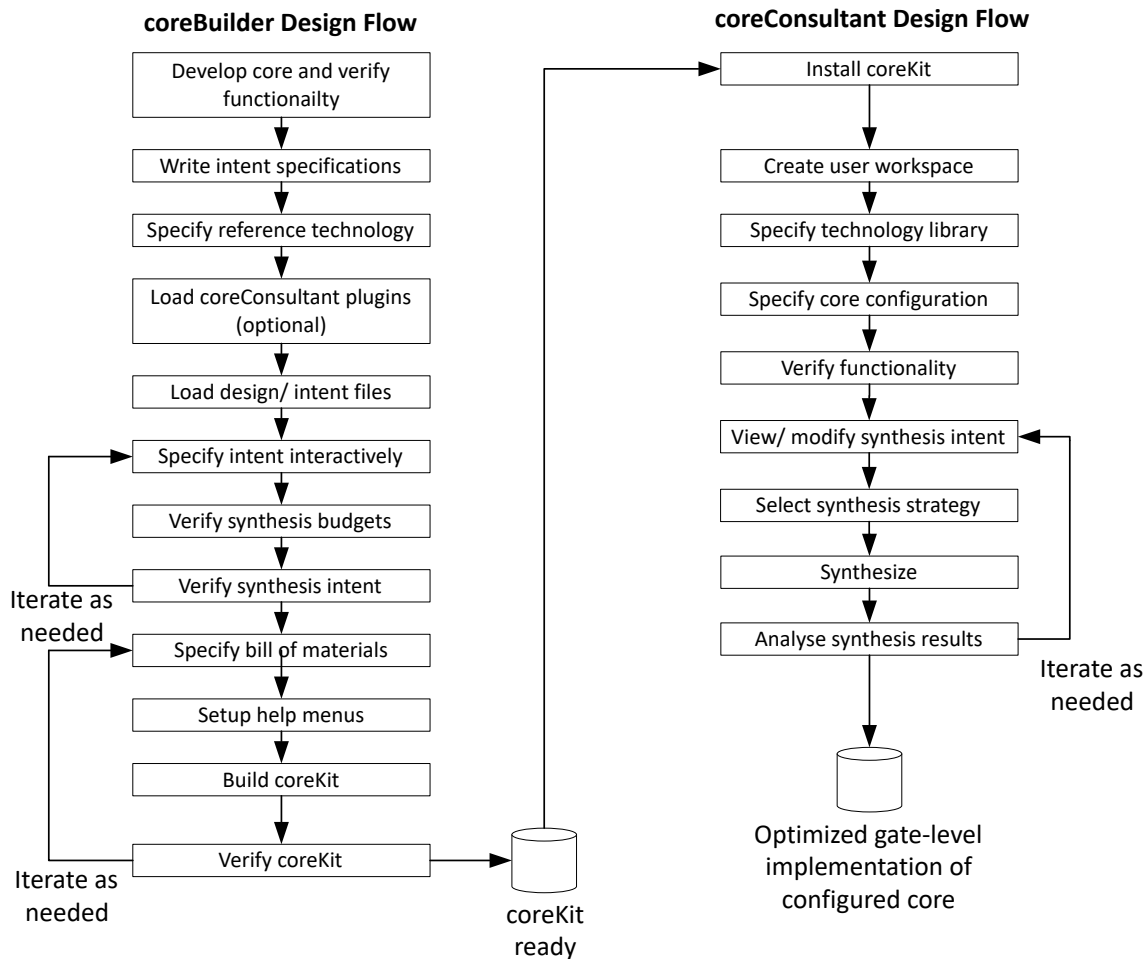
Design knowledge entered by the core developer that describes the design, its characteristics, and core developer's expectation of the context in which the core integrator will implement the core. coreConsultant uses this intent, along with the core integrator's specifications (design context, synthesis strategy, Design Compiler version, and target technology), to guide the core integrator through core configuration and to generate technology and context specific synthesis scripts for the core. An example of configuration intent is a parameter's legal range of values. An example of synthesis intent is whether or not a cell contains muxes and is therefore a candidate for special mux-based optimization. Timing constraints are another example of synthesis intent.

Other unique terms are defined as they occur in this manual. For a complete list of terms and definitions, refer to the glossary.

1.2 coreBuilder/coreConsultant Design Flow

Figure 1-1 illustrates the combined coreBuilder/coreConsultant design flow, with the core developer activities on the left and the core integrator activities on the right.

Figure 1-1 coreBuilder/coreConsultant Design Flow



1.3 System Requirements

The Synopsys coreTools require the following hardware and software resources:

1.3.1 Required Hardware

Section 1.5 of the Release notes talks about the System requirements and supported platforms.

1.3.2 Required Software

coreBuilder requires the following software:

- Netscape or other web browser to view the coreTools online help. Viewing the online coreTools User Guides requires Netscape 3.0 or later. You can download Netscape Communicator for free from <http://www.netscape.com>.
- Adobe Acrobat to view and print the PDF version of the coreTools User Guides. You can download Adobe Acrobat for free from <http://www.adobe.com>.

Implementing the core with coreConsultant requires the following Synopsys software:

- Design Compiler (Expert or Ultra)
- Physical Compiler (optional)
- (V)HDL Compiler
- Test Compiler (optional, for scan insertion synthesis)
- Power Compiler (optional, for clock gating)
- PrimeTime (optional, for timing model extraction)
- Formality (optional, for formal verification)
- Simulation tool (optional for simulation of cores). If you are including custom simulation activities in the coreKit, you will need to verify each custom simulation activity with each supported simulation tool.
- Synplicity (optional for FPGA Synthesis)

2

coreTools Graphical Interface Fundamentals

This chapter describes the basic tasks you need to perform to run the coreTools in graphical mode. It also explores the coreTools menu-driven core packaging flow, and the command-line entry window.

Fundamental tasks include how to start and quit the program and how to run it remotely. Since the coreBuilder IP packaging flow is task driven, each task is covered according to its execution order. In addition to using the menu-driven approach, the Command Window allows you to enter coreBuilder commands in a text field and see the results.

If you plan to run the tutorial by using coreBuilder in GUI mode - which is the recommended approach for first-time users - read this chapter before you begin the tutorial exercises covered in [Chapter 3, “Setting Up the Tutorial”](#) and the chapters that follow it.

This chapter contains the following sections:

- [“Starting coreBuilder”](#) on page 18
- [“Quitting coreBuilder”](#) on page 19
- [“Using the coreBuilder Console”](#) on page 20

2.1 Starting coreBuilder

Before launching coreBuilder, set up the tutorial as described in [Chapter 3, “Setting Up the Tutorial”](#).

During the tutorial, always start coreBuilder from the tutorial directory. Starting from this directory causes the variables defined in the tutorial setup file (.synopsys_rt.setup) to be initialized.

2.1.1 Starting coreBuilder in UNIX

If you use the UNIX operating system, invoke coreBuilder from a UNIX shell, either directly on your machine or in a shell created on the remote host. To start coreBuilder in UNIX,

1. Change to your tutorial directory.
2. Type the following command in your command window:

```
% coreBuilder &
```

The coreBuilder console window opens.

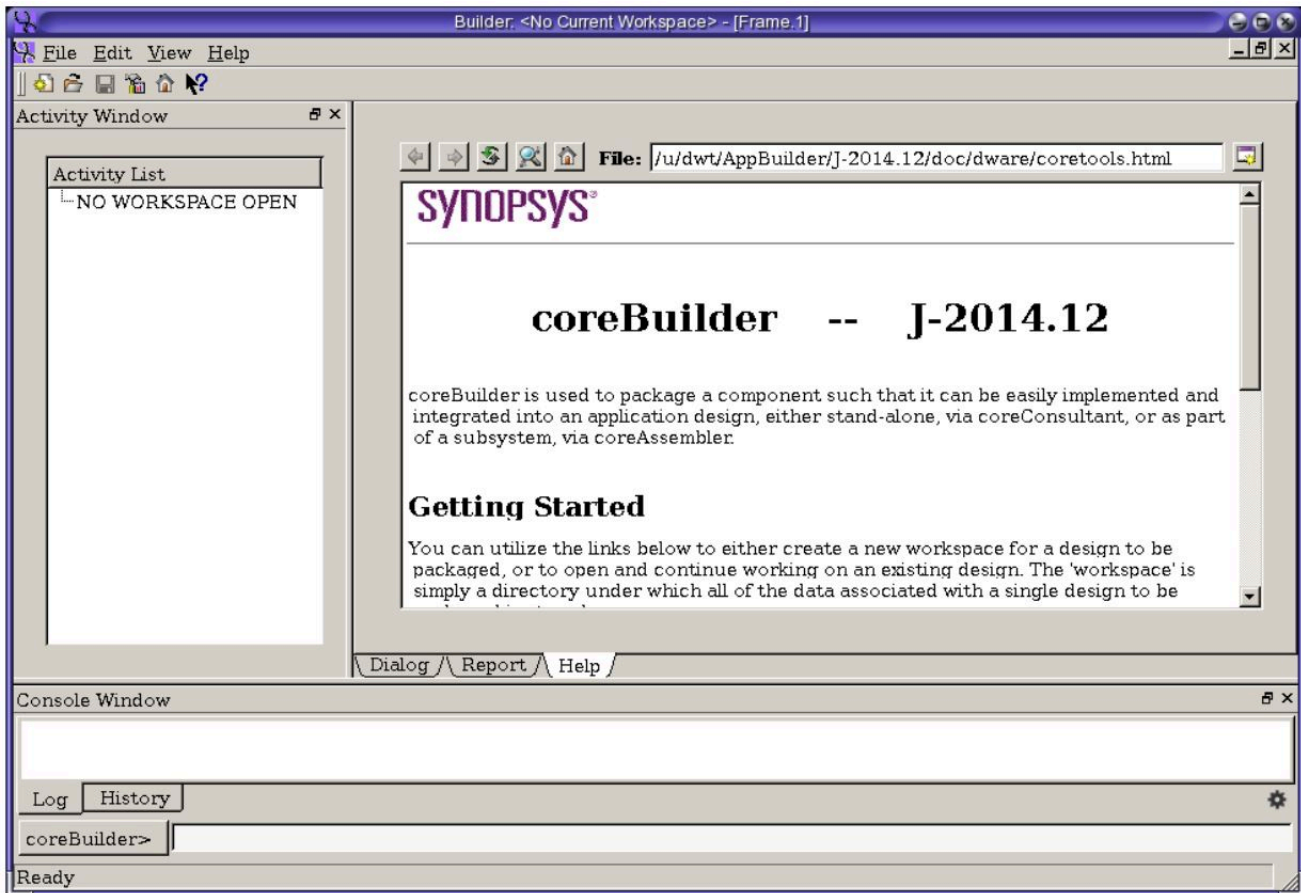
2.2 Quitting coreBuilder

You can quit coreBuilder anytime. Do this in one of two ways: Enter quit in the coreBuilder Command Window text field. or Choose **File > Exit**. When you exit coreBuilder, licenses checked out for your coreBuilder session are automatically checked back in.

2.3 Using the coreBuilder Console

When you start coreBuilder in GUI mode, the following window appears, as shown in [Figure 2-1](#).

Figure 2-1 coreBuilder console window



The coreBuilder console window consists of the following:

■ Menu Bar

The 'File' menu contains workspace manipulation controls. The 'View' menu controls the layout of the console allowing the user to enable/disable the 'Toolbar' and 'Status Bar'.

■ Toolbar

This is the row of menu buttons below the Menu Bar. From left to right, these buttons allow you to:

- ☐ create a new workspace
- ☐ open an existing workspace
- ☐ save the current workspace
- ☐ get some information about the current version of coreBuilder being used
- ☐ access some help for clicked buttons, menus and windows

■ Status Bar

Displays information about the current menu command being executed.

2.3.1 Using the coreBuilder console command line

Use the command line to:

- Enter coreBuilder commands

You can enter Tcl mode commands in the command line text field in the main part of the window.

- View information about commands which are executing during a coreBuilder session.

The Console Window echoes commands and reports generated through coreBuilder menus. Your coreBuilder session also writes Command Window information to the `rt_shell_command.log` file.

2.3.1.1 Entering command line commands

The command-line text field in the Console Window functions exactly like the coreBuilder command line. You can enter any Tcl mode command supported by coreBuilder. For more information on running coreBuilder Tcl mode commands, see the coreBuilder User Guide.

3

Setting Up the Tutorial

This chapter introduces the structure of the tutorial and how to setup the tutorial for the UNIX operating system.

Before you begin the tutorial exercises described in the following chapters, you must setup the tutorial. You do this by creating your tutorial directories and then copying and modifying coreBuilder initialization files.

This chapter contains the following sections:

- [“How This Tutorial Is Organized”](#) on page 24
- [“Creating Tutorial Directories”](#) on page 25
- [“Tutorial Directory Contents”](#) on page 26

3.1 How This Tutorial Is Organized

The tutorial presented in the remainder of this book guides you through the process of packaging a simple Peripheral design. This chapter comprises Part II of the coreBuilder Tutorial. The Appendix at the end of this document contains listings of script files whose commands replicate the tasks covered in that chapter. The Appendix also contains example annotated HDL source code.

3.2 Creating Tutorial Directories

This section describes how to setup the tutorial directories in UNIX.

The tutorial files are located in the coreBuilder installation directory at your site.

3.2.1 Creating Tutorial Directories in UNIX

The files needed for the tutorial are in the <coreTools install path>/doc/dware/tutorial/ directory. To create a tutorial directory under your home directory and copy all subdirectories and files from the Synopsys tutorial directory,

1. Change to your home directory (or preferred working directory for this tutorial).
2. Enter the following command:

```
% cp -r <coreTools install path>/doc/dware/tutorial .
```
3. Check the contents of the tutorial directory and subdirectories against the directories and their files listed in "[Tutorial Directory Contents](#)" on page 26.

3.3 Tutorial Directory Contents

Following are the coreBuilder tutorial directory contents. Compare these lists with your directories to verify that you have the files you need for the tutorial.

3.3.1 <Header> src Directory

This directory contains the verilog source to be packed.

- my_periph_cc_constants.v
- my_periph.v

3.3.2 <Header> pkg Directory

This directory contains the plugin scripts.

3.3.3 doc Directory

The doc directory contains the User Guide for the peripheral design.

4

Preparing the Core for Packaging

This chapter explains how to package the `my_periph` IP in coreBuilder. To make the core configurable the RTL needs to be annotated by introducing reuse-pragmas.

In the first part of this chapter, you will be guided through the process of annotating a number of configuration parameters for the sample RISC CPU core. The next chapter will deal with the creation of a workspace for this core in coreBuilder, and the verification of your configuration parameters.

This chapter contains the following sections:

- [“Annotating HDL Source Files”](#) on page 28

4.1 Annotating HDL Source Files

Before you read your HDL source files, you need to add some coreBuilder-specific annotations to the HDL files. There are three main purposes for HDL source file annotations:

- Enable text substitution so that coreConsultant can write out configured HDL source.
- Specify configuration intent. Synopsys recommends specifying configuration intent through HDL source file annotations and specifying synthesis-related intent through intent command files.
- In addition, you can specify synthesis-related intent through HDL source file annotations. Synopsys recommends using intent command files to specify synthesis-related intent.

4.1.1 Adding RTL Configuration Information

In `my_periph`, all parameters are defined in the file `my_periph_cc_constants.v`. The parameters have a `reuse-pragma` comment line which introduces configurability.

Syntax: `// reuse-pragma attr <Attribute> <Value>`

4.1.2 Understanding the Annotations

The parameter `'APB_DATA_WIDTH'` has the following annotations applied on it.

Table 4-1 APB_DATA_WIDTH Annotations

Annotation	Description
<code>//reuse-pragma attr Description Width of APB Data Bus to which this peripheral is attached</code>	Description associated with the macro. This is used to generate documentation for the parameter.
<code>//reuse-pragma attr Label APB Data Bus Width</code>	Label to be used for this parameter in coreTools GUI.
<code>//reuse-pragma attr EnumValues 8 16 32</code>	The parameter can only accept 8, 16 and 32 as the legal values.
<code>//reuse-pragma attr ReadOnlyParam true</code>	The parameters value is read-only and cannot be modified by the user.

The parameter `'PERIPH_ADDR_SLICE_RHS '` has the following annotations applied on it.

Table 4-2 PERIPH_ADDR_SLICE_RHS Annotations

Annotation	Description
<code>//reuse-pragma attr GroupName HiddenParams</code>	The parameters associated with this group will be grouped together in the GUI and in the reports.
<code>//reuse-pragma attr Visible False</code>	The parameter is not visible in the GUI
<code>//reuse-pragma attr DefaultValue {[calc_addr_rhs @APB_DATA_WIDTH]}</code>	The DefaultValue of the parameter is computed using the function <code>calc_addr_rhs</code> which is defined in the <code>pkg</code> folder.

The parameter `'PERIPH_RDATA_WIDTH '` has the following annotations applied on it.

Table 4-3 PERIPH_RDATA_WIDTH Annotations

Annotation	Description
//reuse-pragma attr MinValue 8	The minimum acceptable value for the parameter is '8'
//reuse-pragma attr MaxValue @APB_DATA_WIDTH	The maximum acceptable value for the parameter is the value of parameter 'APB_DATA_WIDTH'.

5

Create a Workspace and Verify Configuration Intent

The next stage is to create a coreBuilder workspace and load the RISC CPU core files. The configuration intent (as specified in the previous section) can then be verified.

This chapter contains the following sections:

- [“Starting coreBuilder and Creating a Workspace”](#) on page 32
- [“Load Consultant Plugins”](#) on page 33
- [“Load Designs”](#) on page 34
- [“Design Configuration Intent”](#) on page 35
- [“Define Memory Maps”](#) on page 36

5.1 Starting coreBuilder and Creating a Workspace

Go to your chosen working directory, and type

```
%coreBuilder &
```

When the coreBuilder console opens, select File->New from the menu. Leave all of the defaults unchanged, and click 'OK'. The Builder Activity List window will now open, with the 'Setup' tab selected. Before loading the source code for this design, it is necessary to perform a couple of preliminary setup steps.

Refer Chapter 2, "The coreBuilder Environment" in the *coreBuilder User Guide* for additional information.

5.2 Load Consultant Plugins

In this activity we load the plugin directories.

Select the activity 'Load Consultant Plugins' from the activity tree view.

- Click on the add entry icon to add a entry to the directory search path dialog.
- Click on the folder icon to browse and select the directory which contains the plugin.
 - For this tutorial browse and select the 'pkg/cC_plugin' folder.
- Click on the 'Apply' button to complete the activity.

5.3 Load Designs

The next stage in the packaging process is to load the RTL. Select the 'Load Designs' activity from the Activity List.

- In the 'Options' group, click on Input language and select 'Verilog' from the drop down menu. There are no include files so you can leave the 'Includes' box empty.
- In the 'Source Files' group click on the "Append to 'HDL Files' From Directory" button. This will open a directory browser select the 'src' folder and click OK. This will populate all the source files which make up the my_periph design.
- Use the (Up Arrow) and (Down Arrow) icons to reorder the source files such that files are analyzed in the correct order.
- Select the 'my_periph_cc_constants.v' file and click the (Up Arrow) icon.
- Click on the 'Apply' button to complete the activity

5.4 Design Configuration Intent

Once all the HDL source has been loaded, you can verify your configuration annotations. When you click on the 'Design Configuration Intent' window a new window will be opened which contains a spreadsheet showing all HDL parameters and their attributes.

Verify that all of the parameters which you defined in the "Add RTL Configuration Information" section are listed here, and have the appropriate attributes.

To see how the configuration dialog will look when viewed by the coreKit user, click on 'Preview User Configuration Dialog'. Check the operation and appearance of the Configuration Dialog menus, then click 'Close'. Click 'OK'.

5.5 Define Memory Maps

In this activity we specify the memory map for the IP.

Select the activity 'Define Memory Maps' from the activity tree view.

- Select the 'Add Memory Map' icon and enter the memory Map Name.
- Select 'Add Address Block' and enter the details
- Select 'Add Register' and add the details for the register.
- Select 'Add Register Field' and add the details for the register field.

The memory Map after the following steps will look as shown in this screen capture:

Figure 5-1 Memory Map

Name	MemoryAccess	AddressOffset	BitAddressOffset	RegisterSize	RegisterName
MP mmAP	read-write				
BL BLK1	read-write		0		
RG REG1	read-write	0x0		10	
RA RF1	read-write		0	2	
RA RF2	read-write		6	3	

- Click on the 'Apply' button to complete the activity.

You can select the Report tab "Component Register" report for the memory map.

The coreBuilder commands to create the above memory Map in batch mode are:

```
create_memory_map mmAP
create_address_block BLK1 -map mmAP -width 32 -range 0x10 -base_address 0x0
create_register REG1 -block mmAP/BLK1 -offset 0x0 -size 10
create_register_field RF1 -register mmAP/BLK1/REG1 -offset 0 -size 2
create_register_field RF2 -register mmAP/BLK1/REG1 -offset 6 -size 3
autocomplete_activity DefineMemoryMaps
```

6

Synthesis Setup

One of the key advantages of coreBuilder is the ability for the developer to capture the core synthesis intent, along with the packaged core design. The 'Synthesis' stage of the core packaging flow allows the developer to capture a comprehensive set of design and synthesis attributes. This chapter guides you through the process of capturing the core 'synthesis intent'.

This chapter contains the following sections:

- ["Specify Clocks and Resets"](#) on page 38
- ["Specify Timing Exceptions"](#) on page 39
- ["Specify Port Constraints"](#) on page 40
- ["Verify Budgets"](#) on page 41

6.1 Specify Clocks and Resets

Click the 'Synthesis' tab. Note that the Technology Setup activity is already completed. This activity is provided for cores which are technology-dependent, so that you may setup your synthesis intent main reference to actual library cells etc. In this case, you do not need to complete this activity (the my_periph core is technology-independent).

Click on 'Specify Clocks'. The current design will be set to the top level design by default (in this case 'my_periph').

To define the clock(s) for the design follow the steps below:

- Click on 'Add...'. The my_periph design has one clock - 'pclk'.
- Select the 'Real Clock' from the Add Clock dialog and click 'Next'.
- Select 'pclk' from the list of input ports and click 'OK'.
- Click on 'Finish' button to add the clock.

The selected clock source will now appear on the 'Specify Clocks' window. Since there is only one clock in this design, cancel the 'Add Clock' window.

- In the 'Cycletime' window, enter "100ns" to indicate that the clock frequency is 10MHz.
- Click 'Apply' to complete the 'Specify Clocks' activity.

6.2 Specify Timing Exceptions

In order to appropriately direct timing optimization, it is crucial to correctly specify all timing exceptions in your design. Click on the 'Specify Timing Exceptions' activity in the Builder Activity List to open the timing exceptions dialog box.

In the my_periph design, there is only one timing exception - a false path from the asynchronous reset input port. This timing exception is specified as follows:

- set the current design to 'my_periph'
- set the type of timing exception to 'set_false_path'
- in the 'from' box, enter presetn
- click 'New' to add this timing exception to the list at the top of the dialog
- click 'Apply' to complete this activity

6.3 Specify Port Constraints

This activity allows you to specify timing constraints for the I/O ports of your design.

6.3.1 I/O Timing

Since this is defined relative to a clock, and since the developer will not always know the target frequency of the end application, this should ideally be specified in a flexible fashion. To do this, port intent is specified in terms of formula. The steps below describe this in more detail:

- Click on the 'Specify Port Intent' activity in the Builder Activity List
- Select the 'Inputs' tab (if not already specified)

A spreadsheet is presented listing all input ports. Since we are only concerned with maximum timing optimization in this tutorial, you will only be dealing with MaxInputDelay constraints. You should also notice that the clock port specified earlier is not listed here, since an input delay on a clock port is irrelevant.

- Click on the text area under the 'MaxInputDelay[clk] (ns)' column for the 'paddr' input port.
- Key in '=percent_of_period 40'.
- Now specify the maximum input delay as a percentage of the clock cycle time. In this example, use 40 to indicate that the maximum input delay is 40% of the clock period.
- Repeat the same for all Input and Output ports.
- Click 'Apply'.

6.4 Verify Budgets

Before proceeding, it is necessary to verify the correctness of the synthesis intent. There are two elements of budget verification:

- Completeness -- Ensure that all constraints are complete (all I/Os constrained, all clocks have a period defined etc.).
- Consistency -- Ensure that budgets are consistent (all electrically equivalent ports have equivalent constraints, all clock periods must be greater than their associated uncertainties, I/O delays etc.).

Select both 'Report Budget Completeness' and 'Report Budget Consistency', and click 'OK' to execute the checks. The report will be displayed in a browser window. You will notice that there are missing minimum constraints (because we didn't specify them for this example). At this stage it would be possible to go back and fix the incorrect constraints and repeat the budget check.

7

Attach Interfaces

For cores that have similar interfaces, you can create interface definitions that enable coreAssembler to automatically build, connect, and synthesize subsystems that use those cores. This chapter describes how to attach these interfaces, and thereafter do port and parameter association.

This chapter contains the following sections:

- [“Load Interface Definitions”](#) on page 44
- [“Add Interfaces”](#) on page 45

7.1 Load Interface Definitions

To load the interface definition into your coreBuilder workspace, execute the **Load Interface Definitions** activity.

The AMBA interface definitions can be found in the coreTools installation at:

`<coreTools Installation>/auxx/dware/interface_defns`

For this tutorial we will use the AMBA interface definition (`DW-interface_definitions.tcl`)

To complete the **Load Interface Definitions** activity in GUI mode:

1. Click **Packaging > SoC Assembly Intent > Load Interface Definitions** coreBuilder Activity to display the Load Interface Definitions dialog in the Activity View pane.
2. Click on the **Add 'Interface Definitions' From Folder** button.
3. In the "Import Directory" dialog, navigate to and select the folder that contains your interface definition(s). coreBuilder loads the interface definition file(s) in that directory in the 'Interface Definitions File Names' spreadsheet.
4. Click **Apply** to load these definitions, and then move to the **Add Interfaces** activity.

7.2 Add Interfaces

In GUI mode, you instantiate an interface through the 'Add Interfaces' dialog box.

To create the interface instance in GUI mode:

1. Click **Add Interfaces** in the Packaging section of the coreBuilder Activity List to invoke the Add Interfaces dialog box in the Activity View pane.
2. Click **Add** to invoke the 'Add Interface Instance to Core' dialog box.
3. Select an interface from the Interface Definition list. The Interface Definition list includes the interface definitions that you loaded in the **Load Interface Definitions** activity.
4. Enter an instance name in the Instance Name field.
5. Select the Interface Type (see Table 14-59 of the coreBuilder User Guide).
6. Click Apply to add the interface instance and return to the Add Interfaces dialog.

For this tutorial, we will add three interfaces to the core:

Table 7-1 Interfaces to Be Added In Tutorial

Interface	Interface Definition	Interface Name	Interface Type
Interface 1	W-SoC::APB-AClock	APB_Clock	Consumer
Interface 2	DW-SoC::APB-Reset	APB_Reset	Consumer
Interface 3	DW-SoC::APB-Slave	APB_Slave	Consumer

7.2.1 Port and Parameter Association

Once the interfaces are attached, you must associate the interface ports and parameters with the design ports and parameters.

1. Open the 'Add Interfaces' dialog box, if not already open, and select an instance from the list of interface instances.
2. Click **Port + Parameter Setup...** to invoke the 'Associate Interface Ports and Parameters' dialog box, where you can specify how to link the interface ports and parameters to the design ports and parameters.
3. For interface ports and parameters that do not correspond to design parameters, but still require association, specify <open> as the value for the 'Link to' field.

For this tutorial, perform the following port and parameter associations as shown in [Table 7-2](#) on page 45 and [Table 7-3](#) on page 46, respectively.

Table 7-2 Port Associations

Interface Instance	Interface Port	Design Port
Interface Instance APB_Clock	pclk	pclk
Interface Instance APB_Reset	presetn	presetn

Table 7-2 Port Associations (continued)

Interface Instance	Interface Port	Design Port
Interface Instance APB_Slave	paddr	paddr
	penable	penable
	prdata	prdata
	psel	psel
	pwwdata	pwwdata
	pwwrite	pwwrite
	All other interface ports	<open>

Table 7-3 Parameter Associations

Interface Instance	Interface Parameter	Design Parameter
Interface Instance APB_Slave	DATAWIDTH	APB_DATA_WIDTH
	All other Interface Parameters	<open>

Once the port and parameter association task is complete, click on the **Report** button in the 'Add Interfaces' dialog box to view the port and parameter associations.

Click on **Apply** to complete the activity.

8

Packaging

This is the final set of activities which prepare the coreKit for delivery of the core and its associated files and intent. This section will guide you through the process of packaging the RISC CPU core.

This chapter contains the following sections:

- [“Build coreKit” on page 48](#)

8.1 Build coreKit

The final stage in the coreBuilder packaging flow is to build the coreKit for delivery to the core user.

1. Click '**Build coreKit**' in the main Builder Activity List window.
2. Click '**Apply**' because we do not need to add any additional information for this example.

You have now completed packaging of the my_periph IP.

A

Batch Script

You can use the `write_batch_script` command to write out a TCL batch script which can be used to recreate the coreBuilder workspace.

```
coreBuilder> write_batch_script cB.tcl
```

Example A-1 Batch Script Example

```
create_workspace -name tutorial
set_activity_parameter LoadConsultantPlugins PluginDirectories
tutorial/pkg/cC_plugin
autocomplete_activity LoadConsultantPlugins
set_activity_parameter LoadDesigns InputLanguage Verilog
set_activity_parameter LoadDesigns IntentSearchPath {tutorial/src/}
set_activity_parameter LoadDesigns InputFileNames
{tutorial/src/my_periph_cc_constants.v tutorial/src/my_periph.v}
autocomplete_activity LoadDesigns
current_design my_periph
set_parameter_attribute PERIPH_RDATA_WIDTH MinValue 8
set_parameter_attribute PERIPH_RDATA_WIDTH MaxValue =@APB_DATA_WIDTH
set_parameter_attribute PERIPH_RDATA_WIDTH Label {Read data bus width of
peripheral}
set_parameter_attribute PERIPH_RDATA_WIDTH Description {Read Data bus size for
the remap & pause}
set_parameter_attribute PERIPH_RDATA_WIDTH GroupName BasicConfig
set_parameter_attribute APB_DATA_WIDTH MinValue -2147483648
set_parameter_attribute APB_DATA_WIDTH MaxValue 2147483647
set_parameter_attribute APB_DATA_WIDTH EnumValues {8 16 32}
set_parameter_attribute APB_DATA_WIDTH Label {APB Data Bus Width}
set_parameter_attribute APB_DATA_WIDTH Description {Width of APB Data Bus to
which this peripheral is attached}
```

```
set_parameter_attribute APB_DATA_WIDTH GroupName BasicConfig
autocomplete_activity DesignConfigurationIntent
# Memory Maps - DefineMemoryMaps
create_memory_map mmAP
create_address_block BLK1 -map mmAP -width 32 -range 0x10 -base_address 0x0
create_register REG1 -block mmAP/BLK1 -offset 0x0 -size 10
create_register_field RF1 -register mmAP/BLK1/REG1 -offset 0 -size 2
create_register_field RF2 -register mmAP/BLK1/REG1 -offset 6 -size 3
autocomplete_activity DefineMemoryMaps
current_design -quiet my_periph
set_design_attribute {dc_shellVariable[hdlin_vrlg_std]} 2005
set_port_attribute pclk ClockName pclkset_clock_attribute pclk CycleTime 100
set_port_attribute paddr {MaxFallInputDelay[pclk]} {=percent_of_period 40}
set_port_attribute paddr {MaxRiseInputDelay[pclk]} {=percent_of_period 40}
read_sdc -script { set_false_path -from [get_ports presetn]}
set_activity_parameter LoadInterfaceDefinitions InterfaceDefinitionFileNames
"$::synopsys_root/auxx/dware/interface_defns/DW-interface_definitions.tcl"
autocomplete_activity LoadInterfaceDefinitions
set instance [create_interface_instance APB_Slave \
-used {1} \
-type consumer -interface DW-SoC::APB-Slave ]
set_interface_port_attribute -instance $instance \
paddr InterfaceLink paddr
set_interface_port_attribute -instance $instance \
penable InterfaceLink penable
set_interface_port_attribute -instance $instance \
pprot InterfaceLink <open>
set_interface_port_attribute -instance $instance \
prdata InterfaceLink prdata
set_interface_port_attribute -instance $instance \
pready InterfaceLink <open>
set_interface_port_attribute -instance $instance \
psel InterfaceLink psel
set_interface_port_attribute -instance $instance \
pslverr InterfaceLink <open>
```

```
set_interface_port_attribute -instance $instance \  
pstrb InterfaceLink <open>  
set_interface_port_attribute -instance $instance \  
pwwdata InterfaceLink pwwdata  
set_interface_port_attribute -instance $instance \  
pwwrite InterfaceLink pwwrite  
set_interface_parameter_attribute -instance $instance \  
APB3 InterfaceLink <open>  
set_interface_parameter_attribute -instance $instance \  
APB3 ReadOnlyParam 1  
set_interface_parameter_attribute -instance $instance \  
APB4 InterfaceLink <open>  
set_interface_parameter_attribute -instance $instance \  
APB4 ReadOnlyParam 1  
set_interface_parameter_attribute -instance $instance \  
BigEndian InterfaceLink <open>  
set_interface_parameter_attribute -instance $instance \  
DataWidth InterfaceLink <open>  
set instance [create_interface_instance APB_Reset \  
-used {1} \  
-type consumer -interface DW-SoC::APB-Reset ]  
set_interface_port_attribute -instance $instance \  
presetn InterfaceLink presetn  
set instance [create_interface_instance APB_Clock \  
-used {1} \  
-type consumer -interface DW-SoC::APB-Clock ]  
set_interface_port_attribute -instance $instance \  
pclclk InterfaceLink pclclk  
autocomplete_activity AddInterfaces  
autocomplete_activity BuildcoreKit
```


B

Annotated HDL Source Code

This Verilog source code represents the annotated version of the tutorial parameter file (my_periph_cc_constants.v).

Example B-1 Verilog Source for my_periph_cc_constants.v

```
//reuse-pragma attr Description Width of APB Data Bus to which this peripheral
is attached
//reuse-pragma attr GroupName BasicConfig
//reuse-pragma attr Sequence 1
//reuse-pragma attr Label APB Data Bus Width
//reuse-pragma attr EnumValues 8 16 32
//reuse-pragma attr ReadOnlyParam true
`define APB_DATA_WIDTH 32

//reuse-pragma attr Description Bits to decode on paddr
//reuse-pragma attr Visible False
//reuse-pragma attr ReadOnlyParam True
//reuse-pragma attr GroupName HiddenParams
`define PERIPH_ADDR_SLICE_LHS 4

//reuse-pragma attr GroupName HiddenParams
//reuse-pragma attr ReadOnlyParam True
//reuse-pragma attr Visible False
//reuse-pragma attr Label Address slice to use from PADDR
//reuse-pragma attr EnumValues 0 1 2
//reuse-pragma attr DefaultValue {[calc_addr_rhs @APB_DATA_WIDTH]}
`define PERIPH_ADDR_SLICE_RHS 0

//reuse-pragma attr Description Bits to decode on paddr
//reuse-pragma attr Visible False
```

```
//reuse-pragma attr ReadOnlyParam True
//reuse-pragma attr GroupName HiddenParams
`define MAX_APB_DATA_WIDTH 32

//reuse-pragma beginAttr Description
//Each corekit has a version number.
//This is relected in the ascii version number which needs to get translated.
// 0 => 48 -> 30
// 1 => 49 -> 31
// 2 => 50 -> 32
// A => 65 -> 41
// B => 66 -> 42
// C => 67 -> 43
//
//Current Version is 2.02* => 32_30_32_2A
//
//reuse-pragma endAttr
//reuse-pragma attr ReadOnlyParam true
//reuse-pragma attr Visible false
`define VERSION_ID 32'h3230342a

//reuse-pragma attr Description Read Data bus size for the remap & pause
//reuse-pragma attr ReadOnlyParam True
//reuse-pragma attr GroupName BasicConfig
//reuse-pragma attr Label Read data bus width of peripheral
//reuse-pragma attr Sequence 11
//reuse-pragma attr MinValue 8
//reuse-pragma attr MaxValue @APB_DATA_WIDTH
//reuse-pragma attr DefaultValue {[calc_rap_rdata_width @APB_DATA_WIDTH]}
`define PERIPH_RDATA_WIDTH 32
```

C

Solution Scripts

This appendix contains TCL scripts.

This appendix contains the following sections:

- “[setup.tcl](#)” on page [56](#)
- “[synthesis.tcl](#)” on page [57](#)
- “[attachInterface.tcl](#)” on page [58](#)

C.1 setup.tcl

This script sets up the following coreBuilder activities described in the tutorial.

- LoadConsultantPlugins
- LoadDesigns
- DefineMemoryMaps

setup.tcl

```
# create workspace named 'tutorial'
create_workspace -name tutorial

# specify the path to the 'plugin directory'
set_activity_parameter LoadConsultantPlugins PluginDirectories
tutorial/pkg/cC_plugin

autocomplete_activity LoadConsultantPlugins

# Load the RTL
set_activity_parameter LoadDesigns InputLanguage Verilog
set_activity_parameter LoadDesigns IntentSearchPath tutorial/src
set_activity_parameter LoadDesigns InputFileNames
{tutorial/src/my_periph_cc_constants.v \
tutorial/src/my_periph.v}
autocomplete_activity LoadDesigns

# specify the memoryMap
create_memory_map mmAP
create_address_block BLK1 -map mmAP -width 32 -range 0x10 -base_address 0x0
create_register REG1 -block mmAP/BLK1 -offset 0x0 -size 10
create_register_field RF1 -register mmAP/BLK1/REG1 -offset 0 -size 2
create_register_field RF2 -register mmAP/BLK1/REG1 -offset 6 -size 3
autocomplete_activity DefineMemoryMaps
```

C.2 synthesis.tcl

This script, `synthesis.tcl`, completes the coreBuilder Synthesis activity for the tutorial core.

The script defines the following Synthesis activities:

- SpecifyClocks
- SpecifyTimingExceptions
- SpecifyPortConstraints

`synthesis.tcl`

```
current_design -quiet my_periph
set_design_attribute {dc_shellVariable[hdlin_vrlg_std]} 2005
# Specify a clock and set the CycleTime for the clock.
set_port_attribute pclk ClockName pclk
set_clock_attribute pclk CycleTime 100
# Specify the Port Constraints for the IP.
set_port_attribute paddr {MaxFallInputDelay[pclk]} {=percent_of_period 40}
set_port_attribute paddr {MaxRiseInputDelay[pclk]} {=percent_of_period 40}
# Specify the timing exceptions for the IP
read_sdc -script { set_false_path -from [get_ports presetn]}
```

C.3 attachInterface.tcl

This script defines the interfaces on the IP and thereafter associates Interface Ports with Design Ports and Interface Parameters with Design Parameters.

The attachInterface.tcl script defines the following Synthesis activities:

- LoadInterfaceDefinitions
- AddInterfaces

attachInterface.tcl

```
# load the interface definition file
set_activity_parameter LoadInterfaceDefinitions InterfaceDefinitionFileNames
"$::synopsys_root/auxx/dware/interface_defns/DW-interface_definitions.tcl"
autocomplete_activity LoadInterfaceDefinitions

# create a APB_Slave interface and associate the design ports with the
interface ports.
set instance [create_interface_instance APB_Slave \
-used {1} \
-type consumer -interface DW-SoC::APB-Slave ]
set_interface_port_attribute -instance $instance \
paddr InterfaceLink paddr
set_interface_port_attribute -instance $instance \
penable InterfaceLink penable
set_interface_port_attribute -instance $instance \
pprot InterfaceLink <open>
set_interface_port_attribute -instance $instance \
prdata InterfaceLink prdata
set_interface_port_attribute -instance $instance \
pready InterfaceLink <open>
set_interface_port_attribute -instance $instance \
psel InterfaceLink psel
set_interface_port_attribute -instance $instance \
pslverr InterfaceLink <open>
set_interface_port_attribute -instance $instance \
pstrb InterfaceLink <open>
set_interface_port_attribute -instance $instance \
pwwdata InterfaceLink pwwdata
set_interface_port_attribute -instance $instance \
```

```
pwrite InterfaceLink pwrite
set_interface_parameter_attribute -instance $instance \
APB3 InterfaceLink <open>
set_interface_parameter_attribute -instance $instance \
APB3 ReadOnlyParam 1
set_interface_parameter_attribute -instance $instance \
APB4 InterfaceLink <open>
set_interface_parameter_attribute -instance $instance \
APB4 ReadOnlyParam 1
set_interface_parameter_attribute -instance $instance \
BigEndian InterfaceLink <open>
set_interface_parameter_attribute -instance $instance \
DataWidth InterfaceLink <open>
# create a APB_Reset interface and associate the design ports with the
interface ports.
set instance [create_interface_instance APB_Reset \
-used {1} \
-type consumer -interface DW-SoC::APB-Reset ]
set_interface_port_attribute -instance $instance \
presetn InterfaceLink present
# create a APB_Clock interface and associate the design ports with the
interface ports.
set instance [create_interface_instance APB_Clock \
-used {1} \
-type consumer -interface DW-SoC::APB-Clock ]
set_interface_port_attribute -instance $instance \
pclk InterfaceLink pclk
autocomplete_activity AddInterfaces
```